



AuPairA.EU

CAHIER DES CHARGES & DOCUMENT DE REPRISE

Plateforme AuPair A.EU

Première plateforme mondiale dédiée aux au pairs africains — mise en relation avec des familles d'accueil en Europe et en Amérique.

Objet du document : présentation complète du projet, état d'avancement et feuille de route de finalisation, destiné à un développeur reprenant le projet.

Version : 1.0 • **Date :** 9 juin 2026 • **Stack :** Next.js 16 / TypeScript / Prisma 7 / PostgreSQL



Sommaire

- | | |
|---|--------------------------------------|
| 1. Présentation du projet | 7. État d'avancement réel |
| 2. Stack technique | 8. Ce qui est implémenté |
| 3. Charte graphique | 9. Ce qui reste à faire |
| 4. Types d'utilisateurs & modèle économique | 10. Feuille de route de finalisation |
| 5. Architecture & arborescence | 11. Installation & démarrage |
| 6. Modèle de données (Prisma) | 12. Points d'attention techniques |
| | 13. Critères de recette |

1. Présentation du projet

AuPair A.EU est une plateforme web de mise en relation entre de jeunes Africains souhaitant devenir au pair et des familles d'accueil situées en Europe et en Amérique du Nord. Elle se positionne comme la première plateforme mondiale spécialisée sur ce segment.

Proposition de valeur

- **Pour les au pairs** : accès à un annuaire de familles vérifiées, création d'un profil détaillé, messagerie directe, accompagnement vers une expérience à l'étranger.
- **Pour les familles** : recherche d'au pairs qualifiés, profils vérifiés, inscription gratuite et illimitée.

Modèle économique

Abonnement **32 € / 30 jours** côté au pair pour débloquer la messagerie. Inscription **gratuite** côté famille. Paiement prévu par carte bancaire / PayPal (Stripe) et Mobile Money (CinetPay) pour le marché africain.

Objectif de la mission de reprise

Le projet dispose aujourd'hui d'un **front-end complet et abouti** (toutes les pages, l'UI, la charte) mais le **back-end est très largement à construire**. La mission consiste à transformer ce prototype d'interface en une **application réellement fonctionnelle** : authentification, base de données réelle, API, messagerie, paiement, emails.

2. Stack technique

Domaine	Technologie	Notes
Framework	Next.js 16 (App Router)	TypeScript strict, React 19
UI	Tailwind CSS 4 + composants custom	@/components/ui/, Radix UI, Lucide React
ORM	Prisma 7 + adapter @prisma/adapter-pg	Client généré dans src/generated/prisma
Base de données	PostgreSQL	Supabase en production
Authentification	NextAuth.js v5 (beta)	Google + Facebook OAuth + Credentials
Messagerie	Supabase Realtime (prévu)	Polling REST en première étape
Paieement	Stripe + CinetPay (prévu)	CB/PayPal + Mobile Money
Emails	Resend (prévu)	Emails transactionnels
Stockage fichiers	Supabase Storage (prévu)	Photos de profil, documents
Hébergement	Vercel	Build avec prisma generate en postinstall
Internationalisation	next-intl	FR / EN (sélecteur présent dans la navbar)
Tests E2E	Playwright	Installé, suite à écrire

3. Charte graphique

Couleurs

#E87722 Principale (orange)

#1A1A2E Secondaire (bleu nuit)

#FFF3E0 Accent (orange clair)

#F5F5F5 Fond gris

Typographie & icônes

- Police** : Inter (Google Fonts)
- Icônes** : Lucide React
- Style** : arrondis (rounded-xl/2xl), ombres douces, beaucoup d'espace

4. Types d'utilisateurs & modèle économique

Rôle	Accès	Conditions
------	-------	------------

AU_PAIR	Profil, recherche de familles, messagerie	Abonnement 32 €/30 j requis pour la messagerie
FAMILLE	Profil, recherche d'au pairs, messagerie	Inscription gratuite, accès illimité
AGENCE	(prévu)	Non développé à ce stade
ADMIN	Back-office complet	Modération, gestion utilisateurs, paiements

5. Architecture & arborescence

```
src/
├── app/
│   ├── (pages publiques)/      → /devenir-au-pair, /tarifs, /faq, /contact...
│   ├── trouver-au-pair/        → Annuaire au pairs
│   ├── trouver-famille/        → Annuaire familles
│   ├── connexion/              → Login (Credentials + Google + Facebook)
│   ├── inscription/            → Register (stepper 4 étapes)
│   ├── dashboard/
│   │   ├── au-pair/            → Tableau de bord, profil, messages, abonnement
│   │   └── famille/            → Tableau de bord (sous-pages à compléter)
│   ├── admin/                  → Back-office (users, modération, paiements)
│   ├── api/                    → Routes API (à construire en majorité)
│   │   └── auth/[...nextauth] → Handler NextAuth + /register (amorçés)
│   ├── blog/                   → Liste + [slug]
│   ├── sitemap.ts / robots.ts → SEO
│   └── layout.tsx               → Layout racine (Navbar + Footer)
├── components/
│   ├── ui/                     → Button, Badge, Card (design system)
│   ├── layout/                  → Navbar, Footer, DashboardLayout, AdminLayout
│   └── home/                    → HeroSection, StatsSection, HowItWorks...
├── lib/
│   ├── prisma.ts                → Client Prisma (adapter PrismaPg) ✓
│   ├── auth.ts                  → Config NextAuth v5 ✓
│   ├── safe-redirect.ts         → Validation anti open-redirect (OWASP) ✓
│   ├── utils.ts                 → cn(), formatDate(), calculateAge()...
│   └── constants.ts             → Pays, langues, prix
├── generated/prisma/            → Client Prisma généré (NE PAS éditer)
├── middleware.ts                → Protection /dashboard et /admin ✓
└── types/next-auth.d.ts        → Extension types (role, id) ✓
```

6. Modèle de données (Prisma)

Le schéma `prisma/schema.prisma` est **complet et prêt à l'emploi** (14 modèles + 9 enums). Particularité Prisma 7 : le `datasource` ne contient pas d' `url` — celle-ci est définie dans `prisma.config.ts` via `process.env.DATABASE_URL`.

Modèles principaux

Modèle	Rôle
<code>User</code>	Compte (email, password hashé, role, isVerified, isActive) + relations

Account / Session / VerificationToken	Tables NextAuth (OAuth, sessions)
AuPairProfile	Profil au pair complet (identité, langues, expérience, disponibilités, photos, statut de modération)
FamilyProfile	Profil famille (localisation, enfants, tâches, rémunération, préférences)
Subscription	Abonnements (montant, devise, moyen de paiement, dates, statut)
Message	Messagerie (sender/receiver, contenu, statut, flag modération)
Favorite / Report / Block	Favoris, signalements, blocages entre utilisateurs
BlogPost / FaqItem / Testimonial	Contenus éditoriaux (bilingues FR/EN)
Notification	Notifications utilisateur
Ticket / TicketMessage	Support client

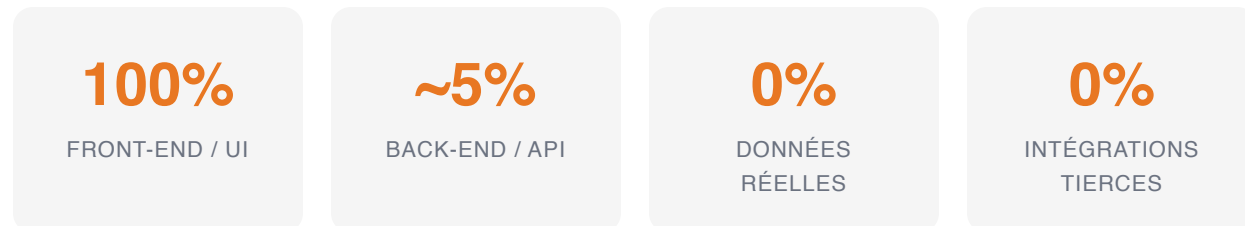
Enums clés

UserRole (AU_PAIR, FAMILLE, AGENCE, ADMIN) · ProfileStatus (PENDING, ACTIVE, HIDDEN, SUSPENDED, DELETED) · SubscriptionStatus (ACTIVE, EXPIRED, CANCELLED, REFUNDED) · PaymentMethod (MOBILE_MONEY, CARD, PAYPAL) · Currency (EUR, XOF, USD) · MessageStatus · MaritalStatus · TicketStatus · TicketPriority .

Important : le schéma est prêt, mais **aucune migration n'a encore été appliquée** à une base réelle, et **aucune page de l'application n'utilise Prisma** à ce jour (toutes les données sont codées en dur dans les composants).

7. État d'avancement réel

Vérification effectuée directement dans le code (et non sur la documentation). Résumé :



Chantier	État	Détail
Pages & UI (22 pages)	FAIT	Toutes les pages publiques, dashboards et admin existent, responsive, charte appliquée
Composants & layouts	FAIT	Design system (Button, Card, Badge), Navbar, Footer, layouts dashboard/admin
Config NextAuth (auth.ts)	FAIT	Providers Google/Facebook/Credentials, callbacks JWT/session, PrismaAdapter
Schéma Prisma	FAIT	14 modèles complets, prêt pour <code>db push</code>
Middleware de protection	FAIT	Garde <code>/dashboard</code> et <code>/admin</code> , redirections sécurisées
Sécurité redirections	FAIT	<code>safe-redirect.ts</code> conforme OWASP (anti open-redirect)
Route API NextAuth	AMORCÉ	<code>/api/auth/[...nextauth]</code> et <code>/api/auth/register</code> créés durant l'analyse
Connexion BDD réelle	À FAIRE	<code>DATABASE_URL</code> = placeholder, aucune migration appliquée
Données dynamiques (Prisma)	À FAIRE	Toutes les pages utilisent des tableaux <code>mock*</code> en dur
API métier (profils, messages...)	À FAIRE	Dossier <code>/api</code> quasi vide
Messagerie fonctionnelle	À FAIRE	UI statique, pas de persistance ni gating réel
Paiement (Stripe / CinetPay)	À FAIRE	Packages installés, zéro intégration ; cités uniquement dans les textes
Emails (Resend)	À FAIRE	Package installé, jamais utilisé
Upload photos (Supabase Storage)	À FAIRE	Bouton décoratif non fonctionnel

Dashboard famille (sous-pages)	À FAIRE	5 sous-pages liées dans la nav mais inexistantes
Tests E2E	À FAIRE	Playwright installé, aucune suite écrite

8. Ce qui est implémenté (détail)

Pages publiques (toutes en place)

/ (accueil), /devenir-au-pair, /accueillir-un-au-pair, /tarifs, /faq, /contact, /trouver-au-pair, /trouver-famille, /blog + /blog/[slug], /mentions-legales, /securite, /sitemap.xml, /robots.txt.

Authentification (UI)

/connexion (email/password + Google + Facebook) et /inscription (stepper 4 étapes choix rôle → infos → profil → confirmation).

Dashboards

Au pair : tableau de bord (KPIs, statut abonnement), /profil, /messages, /abonnement. **Famille** : tableau de bord principal uniquement.

Back-office admin

/admin (KPIs, alertes), /admin/utilisateurs, /admin/moderation, /admin/paiements.

9. Ce qui reste à faire (synthèse)

- Base de données réelle** — provisionner PostgreSQL (Supabase prod + Postgres local en dev), appliquer le schéma, créer un jeu de données de seed.
- Authentification de bout en bout** — finaliser les routes API, brancher inscription/connexion, exposer la session côté client, gérer la déconnexion dans la navbar.
- Remplacement des données mock** — convertir annuaires, dashboards, blog et FAQ en lectures Prisma réelles.
- API métier** — profils (CRUD), messages, abonnement, contact, actions admin.
- Messagerie** — persistance + gating abonnement (côté serveur), puis temps réel.
- Paiement** — Stripe Checkout + webhook, CinetPay Mobile Money.
- Emails transactionnels** — Resend (bienvenue, confirmation, rappels d'expiration).
- Upload de fichiers** — Supabase Storage (photos de profil, pièces d'identité).
- Pages légales** — compléter CGU, CGV, confidentialité, cookies.
- Tests E2E** — suite Playwright sur les parcours critiques.

10. Feuille de route de finalisation

Approche recommandée : **MVP d'abord** (parcours utilisable rapidement), paiement réel et temps réel en dernier. Chaque phase est livrable et testable indépendamment.

Phase 0 — Base de données opérationnelle

- Provisionner PostgreSQL : Postgres local pour le dev (Homebrew/Docker) + projet Supabase pour la prod.
- Renseigner `DATABASE_URL`, puis `npx prisma generate` et `npx prisma db push`.
- Créer `prisma/seed.ts` : 1 ADMIN, 2 au pairs + 2 familles (profils ACTIVE), articles de blog, FAQ, témoignages. Mots de passe hashés via `bcryptjs`.

Phase 1 — Authentification fonctionnelle (bloquant)

- `src/app/api/auth/[...nextauth]/route.ts` exportant `{ GET, POST } = handlers` (amorcé).
- `src/app/api/auth/register/route.ts` : crée `User` + profil (PENDING), hash bcrypt, email unique (amorcé).
- Brancher le stepper d'inscription (remplacer le TODO par un `fetch` POST puis `signIn`).
- Ajouter `<SessionProvider>` dans le layout racine ; brancher l'état connecté + `signOut` dans la Navbar.

Phase 2 — Données réelles : annuaires & profils

- Convertir les annuaires (`/trouver-au-pair`, `/trouver-famille`) en Server Components avec requêtes Prisma (`findMany` où `status = ACTIVE`), filtres via query params.
- Pages de détail `[id]` (lecture + bouton « contacter », incrément `profileViews`).
- Éditeur de profil au pair + famille branchés sur API `/api/profils/...` (GET/PUT).
- Créer les 5 sous-pages manquantes du dashboard famille (profil, recherche, messages, notifications, paramètres).
- KPIs des dashboards + Blog + FAQ alimentés par Prisma.

Phase 3 — Messagerie (polling puis temps réel)

- API `/api/messages` (liste conversations, envoi) et `/api/messages/[userId]` (fil + marquage lu).
- Gating abonnement réel **côté serveur** : helper `hasActiveSubscription(userId)` ; un au pair sans abonnement actif ne peut pas envoyer (les familles ne sont pas restreintes).
- UI branchée avec polling (~5 s), puis migration vers Supabase Realtime.

Phase 4 — Abonnement

- API `/api/abonnement` : activation (création `Subscription` ACTIVE, +30 jours) et lecture (jours restants).
- Étape transitoire possible : activation **simulée** pour tester le gating avant l'intégration paiement réelle.

Phase 5 — Contact & back-office admin

- Formulaire contact → création d'un `Ticket`.
- Brancher les pages admin (utilisateurs, modération, paiements) sur Prisma ; API d'actions (valider / masquer / supprimer) protégées par vérification `role === "ADMIN"`.

Phase 6 — Upload photos (Supabase Storage)

- Bucket Supabase Storage, API `/api/upload` (multipart), stockage des URLs dans `profilePhotoUrl` / `familyPhotoUrl` / `idDocumentUrl`.

Phase 7 — Paiement réel & emails (post-MVP)

- Stripe Checkout + webhook (`/api/webhooks/stripe`) ; CinetPay pour le Mobile Money.
- Resend : emails transactionnels (bienvenue, confirmation d'abonnement, rappels d'expiration).

11. Installation & démarrage

```
# 1. Dépendances
npm install

# 2. Variables d'environnement – copier et compléter
cp .env.local .env # renseigner DATABASE_URL, NEXTAUTH_SECRET, clés OAuth, etc.

# 3. Base de données (schéma déjà prêt)
npx prisma generate      # régénère le client (auto en postinstall)
npx prisma db push       # applique le schéma à la base
npx prisma studio        # (optionnel) interface graphique BDD

# 4. Développement
npm run dev              # http://localhost:3000

# 5. Build production
npm run build

# 6. Tests E2E
npx playwright test
```

Variables d'environnement requises (`.env.local`)

Variable	Usage
<code>DATABASE_URL</code>	Connexion PostgreSQL (pooler Supabase en prod)
<code>NEXTAUTH_URL</code> / <code>NEXTAUTH_SECRET</code>	NextAuth — secret à régénérer pour la prod
<code>GOOGLE_CLIENT_ID/SECRET</code>	OAuth Google
<code>FACEBOOK_CLIENT_ID/SECRET</code>	OAuth Facebook
<code>STRIPE_SECRET_KEY</code> / <code>NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY</code> / <code>STRIPE_WEBHOOK_SECRET</code>	Paiement Stripe
<code>CINETPAY_API_KEY</code> / <code>CINETPAY_SITE_ID</code>	Paiement Mobile Money
<code>RESEND_API_KEY</code> / <code>RESEND_FROM_EMAIL</code>	Emails transactionnels
<code>NEXT_PUBLIC_SUPABASE_URL</code> / <code>..._ANON_KEY</code> / <code>SUPABASE_SERVICE_ROLE_KEY</code>	Realtime + Storage

Toutes les valeurs actuelles sont des placeholders à remplacer par les vraies clés.

12. Points d'attention techniques

Prisma 7

- Le `datasource` du `schema.prisma` n'a **pas** de champ `url` (nouveau v7) ; l'URL est dans `prisma.config.ts`.
- Importer le client depuis `@generated/prisma/client` (et non `@prisma/client`).
- Utiliser l'adapter `PrismaPg` (déjà fait dans `src/lib/prisma.ts`).
- Après tout changement de schéma : `npx prisma generate` puis `npx prisma db push`.

NextAuth v5

- Importer `auth`, `signIn`, `signOut`, `handlers` depuis `@/lib/auth`.
- Dans une route API ou un Server Component : `const session = await auth()`.
- La stratégie de session est **JWT** ; le rôle et l'id sont propagés via les callbacks.

Next.js App Router — règle Client / Server

- Une page qui passe des composants React (ex. icônes Lucide) en props à un Client Component doit être `"use client"`.
- Une page `"use client"` ne peut **pas** exporter `metadata` → créer un wrapper Server Component si besoin de SEO.

Bonnes pratiques attendues : valider toutes les entrées API avant écriture (zod recommandé), réutiliser l'instance Prisma partagée de `src/lib/prisma.ts`, vérifier `role === "ADMIN"` côté serveur sur les routes admin, ne jamais faire confiance au gating uniquement côté UI.

13. Critères de recette (parcours à valider)

#	Parcours	Critère de succès
1	Inscription au pair	Création <code>User</code> + <code>AuPairProfile</code> en base, redirection dashboard
2	Connexion	Credentials + OAuth fonctionnels, redirection selon le rôle
3	Profil	Édition persistée et visible après rechargement
4	Annuaire	Un profil ACTIVE apparaît dans la recherche avec filtres
5	Messagerie — gating	Au pair sans abonnement bloqué (UI + 403 API) ; débloqué après abonnement

6	Messagerie — échange	Le message arrive chez le destinataire
7	Contact	Soumission crée un <code>Ticket</code>
8	Admin	Valider/masquer un profil modifie son affichage public
9	Paieement	Checkout + webhook activent un abonnement réel
10	Build	<code>npm run build</code> passe (TypeScript strict, séparation client/server)